RUNI – Algorithms – Homework 1

Due date: Sunday 09/11/2025

Collaboration policy:

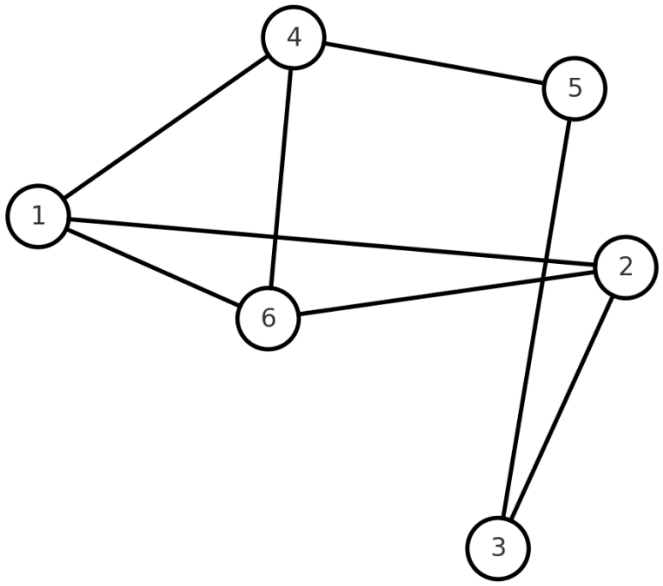
- Each student should submit an individual solution.
- You may discuss homework problems in broad terms with other students.
- You are welcome to discuss the HW with the course stuff in the reception hours.
- You **may not** discuss every last detail, dictate or share written solutions. Do not write your solution together with other students.
- We are going to be strict here. The goal of the homework is to give you an opportunity to struggle with solving problems and formulating your ideas by yourself.

Technical requirements:

- You must fill in your answers on this form in the allocated spaces (or, if you use LaTeX, copy-paste the statements of the problems; no need to fix corrupted formulas). You may add extra space if you need, but too long answers will result in point deduction.
- Submit your solution as a single PDF file of size up to 5MB. The grading app doesn't support other formats and doesn't cope well with large files.
- The page size of your PDF file should be standard A4 and the orientation should be portrait. Before submitting your PDF, open it with Adobe Acrobat Reader, go to File > Properties > Description, and make sure Page Size is around 21 × 29 cm (again, the grading app doesn't support zooming). Please note that scanning A4 pages doesn't guarantee that the resulted PDF file has an A4 page size. Google "resize PDF to A4" if you need.
- Make sure pages are properly ordered and oriented, and text is highly readable: large enough, sharp, in focus, and in high contrast with the background.
- Do not squeeze scanned pages to fit in the allocated spaces; the text becomes too small.
- Do not type in your answers as PDF comments (the grading tool ignores comments).
- Submit your solution electronically via the course Moodle page. After submitting you should download the submitted file back from Moodle to make sure it is the right file.
- Failing to meet any of the technical requirements will result in a fine or your submission not being graded at all.
- For clarifications or any other help, you are welcome to use the course forum in Piazza.

Problem 1:

An undirected graph G is given in the figure.



a. Draw the representation of G by adjacency lists. What is the size of the data structure if it takes 24 bits to store a header node (including a pointer to its adjacency list), and 32 bits to store a cell in a list? (use the actual number of vertices and edges in G to give your answer).

b. Convert G into a directed graph G': Let M be the adjacency matrix of G, then the adjacency matrix M' of G' is defined as follows: M'[i,j]=1 if and only if M[i,j]=1 and i<j.

   i. Does G' contain a directed cycle?

   ii. Repeat part ***a*** of the problem for G'.

c. Let G_1 and G_2 be two DAGs such that $V_1 = V_2 = \{a, b, c, d\}$. It is known that both graphs have the same unique (just one) topological sort. Prove or give a counter example: $E_1 \subseteq E_2$ or $E_2 \subseteq E_1$

Problem 2:

In class we assumed that the topological sort algorithm is applied using a queue – the 'ready nodes' (those having in-deg=0) are enqueued into and dequeued from a queue. In fact, it is possible to use different data structures to keep the ready nodes, in particular, a stack. Here is the algorithm when applied with a stack:

Toposort using a stack:

1. Store each vertex's In-Degree in an array D
2. Initialize a stack with all "in-degree=0" vertices (push vertices with lower index first)
3. While the stack is not empty

   (a) pop and output a vertex

   (b) Reduce In-Degree of all vertices adjacent to it by 1

   (c) push to the stack any of these vertices whose In-Degree became zero (push vertices with lower index first)

4. If all vertices are output then success, otherwise there is a cycle.

Consider the following adjacency matrix M, which represents a directed graph G. $M(i,j) = 1$ if and only if there is an edge from i to j. Empty entries in the figure correspond to $M(i,j) = 0$.

	0	1	2	3	4	5	6
0							1
1				1	1		
2	1						
3							
4				1			
5		1			1		
6						1	

A. Specify the topological sort of G achieved by implementing the topological sort algorithm using a **stack**. In case of a tie, lower index is favored.
B. Analyze the time complexity of this implementation (the analysis should be general, as a function of $|V|$ and $|E|$ and not specific to the given graph). Specify the graph implementation you assume in your analysis (adjacency matrix or lists).

C. Assume the edge (1,2) is added to the graph. List all topological sorts that the resulting graph has or specify a cycle.

Problem 3:

Definition: Let $G = (V, E)$ be a graph. Let $u, v \in V$ be two vertices in the graph. We say that v is reachable from u, if there is a path in G which starts with the vertex u and ends with v.

Definition: A directed acyclic graph $G = (V, E)$ is said to be semi-connected if for any two vertices v and u, v is reachable from u, or u is reachable from v.

Give a linear-time algorithm to determine whether a DAG G is semi-connected. Supply an algorithm, then prove that your algorithm is correct, and then analyze its time complexity

Problem 4:

Let $G = (V, E)$ be a directed graph. Let $G' = (V, E')$ be the graph obtained by reversing the edge directions in G. Formally, $E' = \{(v, u) \mid (u, v) \in E\}$.

Assume that G has exactly 650 different topological sorts. What can be said on the number of topological sorts of G'? Although the instruction is vague, you should supply the tightest possible bounds on the actual answer. Prove the statements you make.

Problem 5:

Let G = (V, E) be a simple undirected graph. The **complement graph** of G is defined as follows: $G^c = (V, E^c)$, where $(u,v) \in E^c$ iff $(u,v) \notin E$. That is, the set of nodes of G^c is the same as that of G, and there is in an edge in G^c connecting any two nodes that do not have an edge connecting them in G.

a. (6 pts.) Describe an undirected graph G with n=4 nodes, such that both G and G^c are connected. Draw both G and G^c.

b. (12 pts.) Prove: For every simple graph with n>2 nodes, if G^c is not connected, then G is connected.